

Singleton în academy

Patru exerciții pe proiectul tău, în ordine.

Cum se lucrează

Patru exerciții, **în ordinea asta**, pe proiectul tău. Începi după ce ai închis B1–B3 din runda 10. Un exercițiu per commit, cu build verde și aplicația pornită înainte de fiecare.

Răspunsurile scrise merg în `RASPUNSURI.txt`, același fișier ca la runda 10.

Teorie

Singleton = o clasă din care există **un singur obiect** în tot programul, iar oricine are nevoie de el îl cere de la clasă, nu îl construiește.

Trei piese, mereu aceleași. Exemplul nu e din academy:

```

public class Setari
{
    private static Setari instance;

    public static Setari Instance
    {
        get
        {
            if (instance == null)
            {
                instance = new Setari();
            }
            return instance;
        }
    }

    private Setari() { }
}

```

Piesa	Ce face
<code>private static Setari instance;</code>	ține unicul obiect; <code>static</code> = aparține clasei, nu unui obiect
<code>public static Setari Instance</code>	îl creează la prima cerere, apoi întoarce mereu același
<code>private Setari()</code>	nimeni din afara clasei nu mai poate scrie <code>new Setari()</code>

Folosire: `Setari s = Setari.Instance;` . Oriunde în program, același obiect.

1. Singleton pe `BookRepository`

Pași:

1. În `Books/Repositories/BookRepository.cs` , adaugă câmpul `private static BookRepository instance;`
2. Adaugă proprietatea `public static BookRepository Instance` , cu `get` ca în teorie.
3. Constructorul `public BookRepository()` devine `private BookRepository()` . Restul liniei (`: base(...)`) rămâne neschimbat.
4. Dă build. Compilatorul îți arată singur unde se mai folosea `new BookRepository()` :

```
ViewStudent.cs(27,45): error CS0122:
'BookRepository.BookRepository()' is inaccessible due to its
protection level
```

5. Repară locul arătat: în loc de `new()` , ia obiectul cu `BookRepository.Instance` .

În `Program.Main` , **temporar** (păstrează-ți `Main` -ul vechi deoparte):

```
BookRepository a = BookRepository.Instance;
BookRepository b = BookRepository.Instance;
Console.WriteLine(ReferenceEquals(a, b));
```

Ieșire așteptată:

```
True
```

Indiciu: `ReferenceEquals` răspunde dacă `a` și `b` sunt **același obiect**, nu doar obiecte la fel. Dacă îți dă `False` , getter-ul tău creează un obiect nou la fiecare cerere: lipsește `if (instance == null)` .

Gata când: ieșirea e `True` , build-ul e verde și ai pus `Main` -ul vechi la loc.

2. Celelalte trei repository-uri

Pași:

1. Fă exact aceiași 3 pași din exercițiul 1 pe `UserRepository` , `CourseRepository` și `EnrolmentRepository` .
2. Dă build. Trebuie să primești **exact** aceste trei erori:

```
ViewLogIn.cs(11,60): error CS0122:
'UserRepository.UserRepository()' is inaccessible ...
ViewStudent.cs(33,37): error CS0122:
'CourseRepository.CourseRepository()' is inaccessible ...
ViewStudent.cs(34,40): error CS0122:
'EnrolmentRepository.EnrolmentRepository()' is inaccessible ...
```

3. Repară fiecare loc arătat cu `XRepository.Instance` .

leșire așteptată: build verde, aplicația pornește și te poți loga ca Ștefan Scarlat.

Indiciu: nu căuta tu locurile cu `Ctrl+F` . `private` pe constructor transformă fiecare `new` rămas într-o eroare de compilare, deci lista de mai sus e căutarea. Dacă primești mai puține sau mai multe erori decât trei, notează în `RASPUNSURI.txt` care și de ce.

Gata când: zero `new XRepository()` în tot proiectul și build verde.

3. Capcana: abonarea dublată

Cu Singleton, un repository trăiește cât tot programul. Exercițiul ăsta îți arată ce se întâmplă cu abonările din constructorul unui ecran.

Pași:

1. **Experimentul.** Pune temporar în `Program.Main` :

```
User u = UserRepository.Instance
    .GetByFirstAndLastName("Stefan", "Scarlat");
new ViewStudent(u);
new ViewStudent(u);

Book carte = new Book(u.Id, "Test", DateTime.Now);
BookRepository.Instance.Add(carte);
BookRepository.Instance.Remove(carte);
```

2. Rulează. **leșire așteptată:**

```
NotificatorTest Book created successfully
NotificatorTest Book created successfully
NotificatorTest Book removed successfully
NotificatorTest Book removed successfully
```

3. În `RASPUNSURI.txt` : de ce apare fiecare mesaj **de două ori**, deși ai adăugat o singură carte?

4. Mută cele trei `Subscribe` (două din constructorul `ViewStudent` , unul din `ViewLogIn`) la începutul lui `Program.Main` și șterge-le din ecrane:

```
BookRepository.Instance.Subscribe(new NotificatorTest());
BookRepository.Instance.Subscribe(new NotificatorFisier(
    Path.Combine("..", "..", "..", "Data", "jurnal.txt")));
UserRepository.Instance.Subscribe(new NotificatorConsola());
```

5. Rulează din nou experimentul, cu abonările de la pasul 4 deasupra lui. **leșire așteptată:**

```
NotificatorTest Book created successfully
NotificatorTest Book removed successfully
```

6. Scoate experimentul. În `Main` rămân abonările și login-ul de dinainte.

Indiciu: `Subscribe` adaugă în lista **obiectului**. Înainte de Singleton, fiecare `ViewStudent` avea repository-ul lui, deci lista lui. Acum e un singur repository: fiecare `new ViewStudent(...)` mai adaugă încă un ascultător în **aceeași** listă.

Gata când: experimentul afișează fiecare mesaj o singură dată, iar ecranele nu mai conțin niciun `Subscribe`.

4. Ecranul de admin

`ViewLogIn` are de mult `//viewAdmin` în locul ecranului de admin. Acum îl scrii.

Pași:

1. Clasă nouă `ViewAdmin`, în rădăcina proiectului, lângă `ViewStudent`.
2. Câmp: `private UserService userService = new(UserRepository.Instance);`
3. `public void Viewer()`: meniu cu `0` = ieșire și `1` = adaugă student, construit ca meniurile din `ViewStudent`.
4. `public void AdaugareStudent()`: citește numele, prenumele, emailul și vârsta, apoi cheamă `userService.CreateStudent(...)`. La succes afișează `Studentul <prenume> <nume> a fost adaugat`. La `AcademyException` afișează `ex.MesajPentruUtilizator`, ca în `ViewStudent`.
5. În `ViewLogIn`, înlocuiește `//viewAdmin` cu deschiderea ecranului:

```
ViewAdmin viewer = new();
viewer.Viewer();
```

Rulare: te loghezi ca `Grozavu / Alex`, cu parola adminului din `users.txt`, apeși `1` și introduci `Ionescu`, `Maria`, `maria.ionescu@gmail.com`, `20`. Apoi apeși din nou `1` și introduci exact aceleași date.

Ieșire așteptată, printre liniile de meniu (prima linie apare imediat după `Varsta:`, pe același rând):

```
NotificatorConsola: User created successfully
Studentul Maria Ionescu a fost adaugat
Studentul "maria.ionescu@gmail.com" e deja in lista.
```

Apoi repornești aplicația și te loghezi ca `Ionescu / Maria` : trebuie să vezi `Logat cu succes!` .

În `RASPUNSURI.txt` : linia `NotificatorConsola: ...` a fost scrisă de un ascultător abonat în `Program.Main` , nu în `ViewAdmin` . Cum a ajuns la el mesajul? Ce s-ar fi afișat dacă `ViewAdmin` ar fi folosit un `new UserRepository()` al lui?

Indiciu: vârsta se citește cu `Int32.TryParse` , ca tastele din meniu. Cu `Int32.Parse` , un text ca `doua` aruncă `FormatException` , care nu e `AcademyException` , deci `catch` -ul tău nu îl prinde și programul se oprește.

Gata când: Maria apare în `users.txt` , se poate loga, iar mesajul `NotificatorConsola` apare la adăugare.

Întrebare finală, în scris

Singleton a rezolvat o problemă reală în proiectul tău. Are și un preț. Uită-te la câmpul din `ViewAdmin` : de unde își ia `ViewAdmin` repository-ul, și cine poate vedea asta din afara clasei? Ce s-ar complica dacă ai vrea să testezi `ViewAdmin` cu un `users.txt` de test?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro